Лабораторная работа № 5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

1. Параметры экспериментов и система

Обучение линейной регрессии с L1-регуляризацией проксимальным градиентным методом реализовано в четырёх вариантах: последовательном, OpenMP (объекты делятся между потоками, частичные градиенты складываются через reduction), SIMD (omp simd на циклах по признакам) и OpenMP + SIMD. Данные сгенерированы с фиксированным seed (признаки в [−1, 1], половина истинных коэффициентов нулевая); в начале w = 0, b = 0, η = 0.5, K = 100, λ = 10^{-3}, если не указано иное. Измерялось только обучение (omp_get_wtime(), медиана из 3 запусков). Intel Core i5-12450H (4 P- и 4 E-ядра, 12 потоков, AVX2, L3 12 МБ), WSL Ubuntu 24.04, GCC 13.3.0, флаги -O3 -march=native -fopenmp (кроме табл. 7).

2. Результаты экспериментов

Таблица 1. Проверка корректности (N = 10^4, d = 32, K = 100, 8 потоков, допуск Δw < 10^{-6})

Реализация	Итоговая функция потерь	Δw	Корректность подтверждена?
Последовательная	0.0096323594	0	да
OpenMP	0.0096323594	$3.5\cdot10^{-18}$	да
SIMD	0.0096323594	$1.1\cdot10^{-16}$	да
OpenMP + SIMD	0.0096323594	$1.1\cdot10^{-16}$	да

Таблица 2. Сравнение реализаций (N = 10^5, d = 32, K = 100, 8 потоков)

Вариант	Время, с	Ускорение S
Последовательная версия	0.232358	1.00
OpenMP	0.068789	3.38
SIMD	0.218048	1.07
OpenMP + SIMD	0.059626	3.90

Таблица 3. Влияние числа объектов N (d = 32, K = 100, 8 потоков)

N	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
10^4	0.010823	0.004408	0.011815	0.002919
10^5	0.253373	0.063692	0.222483	0.060248
10^6	2.437492	0.779689	2.231050	0.742061

Таблица 4. Влияние числа признаков d (N = 10^5, K = 100, 8 потоков)

d	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
8	0.043289	0.008874	0.052638	0.011398
32	0.241305	0.063104	0.218611	0.060368
128	1.034101	0.295348	0.901567	0.293227
256	2.600227	0.645176	2.450004	0.631141

Таблица 5. Влияние числа потоков (N = 10^5, d = 32, K = 100)

p	OpenMP, с	Sp OpenMP	Ep OpenMP	OpenMP + SIMD, с	Sp OpenMP + SIMD	Ep OpenMP + SIMD
1	0.286395	1.00	1.00	0.262606	1.00	1.00
2	0.142960	2.00	1.00	0.135017	1.94	0.97
4	0.086841	3.30	0.82	0.083320	3.15	0.79
8	0.061692	4.64	0.58	0.060004	4.38	0.55
16	0.095417	3.00	0.19	0.087346	3.01	0.19

Таблица 6. Влияние коэффициента регуляризации λ (N = 10^5, d = 32, K = 100, η = 0.5)

λ	Итоговая функция потерь F(w, b)	Число коэффициентов с |wj| < 10^{-6}	Время обучения, с
0	0.0016649187	0	0.252553
10^{-4}	0.0024648347	11	0.249315
10^{-3}	0.0096405207	16	0.257386
10^{-2}	0.0792736615	16	0.262267
10^{-1}	0.5864269381	21	0.269441

Таблица 7. Влияние флагов компиляции (N = 10^5, d = 32, K = 100)

Флаги компиляции	Время, с	Ускорение отн. 1-й строки	Примечания
-O0 -fopenmp	2.036980	1.00	serial, без оптимизаций
-O3 -fopenmp	0.439642	4.63	serial, автовекторизация SSE
-O3 -march=native -fopenmp	0.377434	5.40	serial, автовекторизация AVX2
-O3 -march=native -fopenmp + omp simd	0.351068	5.80	версия simd

3. Векторизация

Отчёт -fopt-info-vec показал, что компилятор сам векторизовал основные циклы во всех версиях, даже без omp simd: подсчёт прогноза, накопление градиента и деление на N. Не векторизовались внешний цикл по объектам (в нём два цикла подряд), обновление весов (есть условия if) и генерация данных (вызов rand()), но на время обучения они почти не влияют. Поэтому -O3 и -march=native ускорили последовательную версию в 5.4 раза, а omp simd добавил всего 8 %.

4. Выводы

Быстрее всех работала версия OpenMP + SIMD: при N = 10^5 и 8 потоках она ускорилась в 3.90 раза, OpenMP – в 3.38 раза, а SIMD в одном потоке – всего на 7 %. SIMD сильнее помогает, когда данные помещаются в кеш (при N = 10^4: 3.71 против 2.46), а при d = 8 даже замедляет программу из-за слишком коротких циклов.

Ускорение меньше числа потоков, потому что на каждой итерации все потоки читают всю матрицу X (25.6 МБ, больше кеша), а память у них общая. Поэтому эффективность падает со 100 % на 2 потоках до 55–58 % на 8, а 16 потоков работают медленнее 8, так как у процессора только 12 потоков. На маленьких задачах ускорение съедают затраты на запуск потоков, на больших – скорость памяти, а сложение частичных градиентов почти не занимает времени. С ростом λ функция потерь растёт и всё больше коэффициентов становятся нулевыми (16 при λ = 10^{-3}, 21 при λ = 10^{-1}), а время обучения не меняется.